GTAE-IDS: Graph Transformer-Based Autoencoder Framework for Real-Time Network Intrusion Detection

Jalal Ghadermazi, Soumyadeep Hore, *Graduate Student Member, IEEE*, Ankit Shah, *Senior Member, IEEE*, and Nathaniel D. Bastian, *Senior Member, IEEE*

***Abstract*—Network intrusion detection systems (NIDS) utilize signature and anomaly-based methods to detect malicious activities within networks. Advances in machine learning (ML) and deep learning (DL) algorithms have enabled NIDS to analyze large volumes of data and identify complex patterns. However, traditional ML/DL approaches in NIDS have primarily relied on flow-based features and utilized flat data formats, such as vectors or grids, which limit their ability to recognize the structural and contextual nuances of network attacks, particularly in real-time. Additionally, most NIDS depend on supervised or semi-supervised learning, requiring extensive labeled data that is time-consuming to generate and not always feasible. This reliance restricts their ability to detect novel attacks, as they typically only recognize threats similar to those encountered during training. Hence, there is a significant need to develop NIDS that can operate in near real-time, eliminate the need for labeled data, and effectively identify novel attack patterns. We propose GTAE-IDS, a novel unsupervised packet-based graph neural network framework aimed at early and precise anomaly detection in network traffic. GTAE-IDS employs graph embeddings to capture and process network traffic data swiftly, creating sequential packet-based graphs that reflect network communications. Our approach employs graph autoencoders to identify structural and global patterns in benign data without needing labeled graph data, enhancing detection capabilities against novel attacks. Incorporating transformers in the encoder segment, GTAE-IDS effectively discerns contextual patterns in network traffic, achieving over 98% accuracy in identifying malicious activities on benchmark network intrusion data sets.**

***Index Terms*—Network intrusion detection, graph representation learning, graph transformer, autoencoders, real-time anomaly detection, network security.**

Received 24 May 2024; revised 22 November 2024; accepted 24 March 2025. Date of publication 3 April 2025; date of current version 22 April 2025. This work was supported in part by U.S. Military Academy (USMA) under Grant W911NF-22-2-0045 and in part by U.S. Army Combat Capabilities Development Command (DEVCOM) U.S. Army Command, Control, Communications, Computers, Cyber, Intelligence, Surveillance and Reconnaissance (C5ISR) Center under Grant USMA21056. The associate editor coordinating the review of this article and approving it for publication was Prof. Husrev T. Sencar. *(Corresponding author: Ankit Shah.)*

Jalal Ghadermazi and Soumyadeep Hore are with the Department of Industrial and Management Systems Engineering, University of South Florida, Tampa, FL 33620 USA (e-mail: jghadermazi@usf.edu; soumyadeep@usf.edu).

Ankit Shah is with the Department of Operations and Decision Technologies, Indiana University, Bloomington, IN 47405 USA (e-mail: ankit@iu.edu).

Nathaniel D. Bastian is with United States Military Academy, West Point, NY 10996 USA (e-mail: nathaniel.bastian@westpoint.edu).

This article has supplementary downloadable material available at https://doi.org/10.1109/TIFS.2025.3557741, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3557741

I. Introduction

NETWORK intrusion detection systems (NIDS) monitor network traffic to detect intrusions through signature-based and anomaly-based detection techniques. The progress in machine learning (ML) and deep learning (DL) algorithms, especially for anomaly-based detection, has significantly enhanced the precision of NIDS in distinguishing between benign and malicious activities in network traffic. The integration of these algorithms within NIDS empower them to process and learn generalizable and intricate patterns from large amounts of data [1]. Until recently, the predominant approach in ML/DL-based network intrusion detection systems (NIDS) relied on flat data formats like vectors or grids, which are commonly used in image representations. Although these models showcase impressive capabilities in learning patterns within the data, they face limitations in capturing the intrinsic structural and contextual patterns associated with network attacks. Network data inherently possesses a graph structure, and graphs serve as universal representations that provide a high-level and abstract overview of a system, encapsulating both structural and contextual information. Hence, representing network data as a graph and developing graph representation learning (GRL) methods to learn the associated structural features present a promising approach. The increasing popularity of DL has facilitated the emergence of graph neural network (GNN) involving spectral and spatial convolutions applied to graph structures, demonstrating promising results in the context of NIDS [2], [3], [4], [5], [6], [7], [8], [9].

However, existing GNN studies in literature face several limitations. GNN-based NIDS typically employ flow-based features, in which packets are aggregated and detection techniques are applied on the characteristics of the flows. Flow-based NIDS are well-suited for offline traffic analysis as they process packets collected after communication has concluded, making them effective only for retrospective assessments. However, intrusions and attacks manifest through sequences of suspicious and benign packets between the sender and receiver in a network. Therefore, a packet-based GNN approach is suitable for real-time traffic analysis, enabling prompt detection of network anomalies as they occur. While many existing GNN studies focus on flow-based features, the exploration of packet-based GNNs for NIDS remains underexplored. Furthermore, a significant portion of GNN-

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

based NIDS follows either a supervised or semi-supervised learning paradigm which introduces two major challenges. Firstly, labeling graph data is a time-consuming process and may not always be applicable, particularly in scenarios like detecting anomalous activity within expansive networks where obtaining ground truth labels proves challenging. Additionally, training supervised models often demand significant computational resources, especially when dealing with vast data sets such as network data [10], [11]. Secondly, these methods struggle in detecting novel attacks, which employ new structural patterns that have not been exposed to the models during training, leading to limitations in the detection capability of GNN-based NIDS.

In this study, we introduce GTAE-IDS, a novel unsupervised packet-based GNN framework utilizing graph embeddings aimed at achieving early and precise anomaly detection in network traffic. The contributions of this study are multi-faceted and encompass the following aspects. First, GTAE-IDS operates in near real-time, capturing network traffic data and decomposing it through dynamic processes to construct sequential packet-based graphs that represent network communications. Second, we propose utilizing graph autoencoders to uncover structural and global patterns within benign data, employing a fully unsupervised approach. This eliminates the requirement for labeled graph data and enhances the robustness of NIDS against previously unseen attacks, bolstering its ability to detect emerging threats. Furthermore, the sequences of packets within each communication graph bear resemblance to the sequences of words in a sentence. Hence, recognizing the sequential nature of packet transmission in each communication, we employ transformers in the encoder segment of the graph autoencoder model. This approach aims to capture the contextual patterns inherent in benign data, thereby enhancing the model's ability to discern meaningful structures within the network traffic data. In summary, although dynamic GNNs are employed in other studies, the unique application and innovations found in GTAE-IDS, specifically the unsupervised learning framework, packet-level representation, integration of transformers, near real-time processing, and emphasis on emerging threats, constitute significant advancements in the realm of network anomaly detection. These innovations enhance detection capabilities and tackle critical challenges present in current methodologies. Finally, our framework effectively identifies ongoing malicious communications with accuracies exceeding 98% on three contemporary publicly available network intrusion data sets, CIC-IDS2017 [12], CIC-IDS2018 [13], and ACI-IoT-2023 [14]. Notably, GTAE-IDS achieves this high level of accuracy by constructing communication graphs with very few initial packets in an ongoing flow, indicating a paradigm shift in the early detection of network intrusions.

The remainder of this paper is organized as follows. Section II examines prior research in network intrusion detection and highlights distinctions between our approach and existing methods. Section III details the design of our proposed framework. Section IV provides the details of the experiments including the data, specifics of the models, and experimental cases. Section V presents the experimental results and demonstrates the effectiveness of our framework. Finally, Section VI presents the concluding remarks of this paper and outlines future research directions.

II. Related Literature

In this section, we present our literature review findings on different graph representations of network data and the data types used to construct graphs for identifying attacks and anomalies in network traffic.

A. Graph Representation of Network Traffic

A computer network can be viewed as a system with multiple entities interconnected. Representing large computer networks as graphs is highly intuitive. Nodes typically represent the entities within the network (such as printers, servers, databases, desktops, etc.), while communication or connections between these entities are usually depicted as edges. Literature studies have explored using extracted flow data, authentication data, and packet data to construct this node-edge structure, illustrating computer network traffic [8], [15], [16].

Flow data is the most popular among them. Flows are abstractions made of multiple packets captured over a period of time. One flow can be identified as a 5-tuple information: Source IP, Destination IP, Source Port, Destination Port, and Protocol. Within a flow, data encapsulates statistical summaries of various packets comprising the flow, including metrics such as average byte length, packet count, and minimum/maximum/average inter-arrival times, among others. Some studies have advocated a straightforward strategy where flows represent all nodes, with edges representing communications between the source and destination IP addresses involved in the respective flow [8]. Initially, a bipartite graph is constructed encompassing all source and destination IP addresses, which subsequently undergoes transformation into a linegraph. This method proposes the incorporation of residual nodes to balance the size disparity between the source and destination node sets. An alternative approach involves creating an intermediary attribute node storing diverse statistical information pertaining to the flow between two IP addresses [15]. The direction of flow also assumes significance in the representation. Chang and Branco amalgamated forward and backward packets to generate an aggregated flow, whereas Pujol-Perich et al. considered two distinct flows for forward and backward packets.

Another efficient way of handling the intrusions is by focusing specifically on authentication requests. This method is particularly effective in detecting attacks that rely on malicious authentications. In this type of representation, nodes are mostly users and edges are analogous to authentication requests [16].

A meticulous method of constructing detailed graphs with exhaustive information involves utilizing packet data. Packets encapsulate payloads, which constitute the actual data intended for transmission, along with several layers of header information. Despite encryption measures, packet payload data can either reveal or obscure crucial attack information. Intuitively, packet data can be translated into graphs [17], [18], for

TABLE I
SUMMARY OF DIFFERENT GRAPH-BASED NIDS IN RECENT LITERATURE

Paper	Learning Type	Classification Task	Graph Type	Algorithm	Data Type	Data Set	Performance
[3]	Supervised	Node	Homogeneous, Unattributed	GCN + FNN	Flow-based	Data from synthetic Botnet topologies	99% accuracy
[4]	Supervised	Node	Homogeneous, Unattributed	GCN + FNN	Flow-based	Data from synthetic Botnet topologies	98% F1
[5]	Supervised	Node	Homogeneous, Unattributed	GIN	Flow-based	Data from synthetic Botnet topologies	99% F1
[6]	Semi-supervised	Node	Heterogeneous. Unattributed	GCN	Flow-based	CTU-13	98.22% micro-F1
[15]	Supervised	Node	Heterogeneous, Unattributed	GNN, GRU	Flow-based	CIC-IDS2017	99% wighted F1
[19]	Supervised	Node	Homogeneous, Attributed	Graph Network	Flow-based	CTU 13 and Synthetic samples	99% AUC
[7]	Supervised	Edge	Heterogeneous, Attributed	Modified GraphSAGE	Flow-based	BoT-IoT, ToN-IoT, NF-BoT-IoT, NF-ToN-IoT	99% F1
[8]	Supervised	Node	Homogeneous, Attributed	E-GraphSAGE, GAT	Flow-based	USNW-NB15, CIC-Darknet2020, ToN-IoT, CIC-IDS2018	98% weighted-F1
[20]	Supervised	Edge	Heterogeneous, Attributed	E-GraphSAGE	Flow-based	UNSW-NB15	99.88% F1
[21]	Unsupervised	Edge	Heterogeneous, Attributed	E-GraphSAGE, Isolation Forest	Flow-based	UNSW-NB15, CIC-IDS2018	96% macro-F1
[22]	Supervised	Graph	Heterogeneous, Attributed	GIN	Flow-based	CIC-IDS2017, CIC-DoS-2017	96.5% F1
[17]	Supervised	Edge	Spatio-temporal, Attributed	ST-GCN	Packet-based	CAIDA+Synthetic samples	90% accuracy
[18]	Supervised	Node	Spatio-temporal, Attributed	Auto Metric GNN	Packet-based	CSIC 2010, ISCXIDS2012	96% F1
[9]	Supervised	Graph	Attributed	TCGNN	Packet-based	UNB-ISCX, TrafficX	98.3% macro-F1
[23]	Supervised	Graph	Attributed	GNN, FEATHER algorithm	Packet-based	CIC-IDS 2017	99% accuracy

example, by assigning nodes to IP addresses and establishing an edge for each packet transmitted between a pair of nodes. However, the practice of creating distinct edges for every packet typically leads to scalability concerns, rendering such a graph structure seldom employed in the literature. Table I provides a summary of the different graph representations and learning-based approaches used in literature for NIDS. We have identified and listed the type of learning, classification task, graph, DL algorithm(s), and data used in each of the studies presented in the respective papers. We provide an extended literature review covering studies on both flow-based and packet-based GNNs for NIDS in the supplemental appendix.

B. Gaps in Literature and Our Work

While flow-based detection methods are commonly utilized in NIDS, their application is primarily restricted to offline detection of network attacks and anomalies. Furthermore, these approaches entail extracting features from packet data and aggregating them to generate arbitrary feature sets, leading to two primary issues: (i) overlooking the intrinsic malicious behaviors of certain attacks inherent in packet data, and (ii) encountering varied sets of training features with differing sizes and complexities, influenced by unique organizational contexts. Conversely, packet-based NIDS offer a granular perspective of the actual data transferred within networks, rendering them suitable for near real-time detection of anomalous activities. Nonetheless, existing packet-based NIDS are significantly constrained, primarily treating network data as attributed graphs within a supervised or semi-supervised learning framework, thus posing challenges both during the training phase and in effectively detecting newly emerged attacks.

Our research leverages packet-based network data to approach intrusion detection through a graph-based framework. In contrast to previous packet-based NIDS approaches, we construct dynamic graphs for each network flow utilizing only the sequence of the initial p packets, rather than the entire flow data. This methodology offers two significant advantages for NIDS. Firstly, it enables quick identification of anomalous activities within the network. Secondly, by exclusively utilizing benign communication graphs, we introduce graph transformer-based autoencoder (GTAE) for unsupervised detection of anomalous activities, eliminating the dependency on labeled data and enhancing resilience against novel attacks. GTAE autonomously extracts embeddings from both graph features and packet data through the utilization of the graph transformer-based encoder model. We then employ a selection of ML/DL algorithms tailored for anomaly detection, which obviates the need for extensive labeled training samples and manual parameter tuning. Next, we describe our methodology in detail.

III. METHODOLOGY

Network traffic can be effectively modeled as a dynamic graph based on two dynamic processes, wherein the graph structure undergoes variations in each process. The first process is time-based, causing changes in the graph structure over time. The number of nodes (vertices) and edges in the graph associated with the network traffic changes for different timestamps $\{t, t+1, \ldots, t+n\}$. The second dynamic process is event-based and is contingent upon the number of packets, p, for each communication in the network traffic data at any

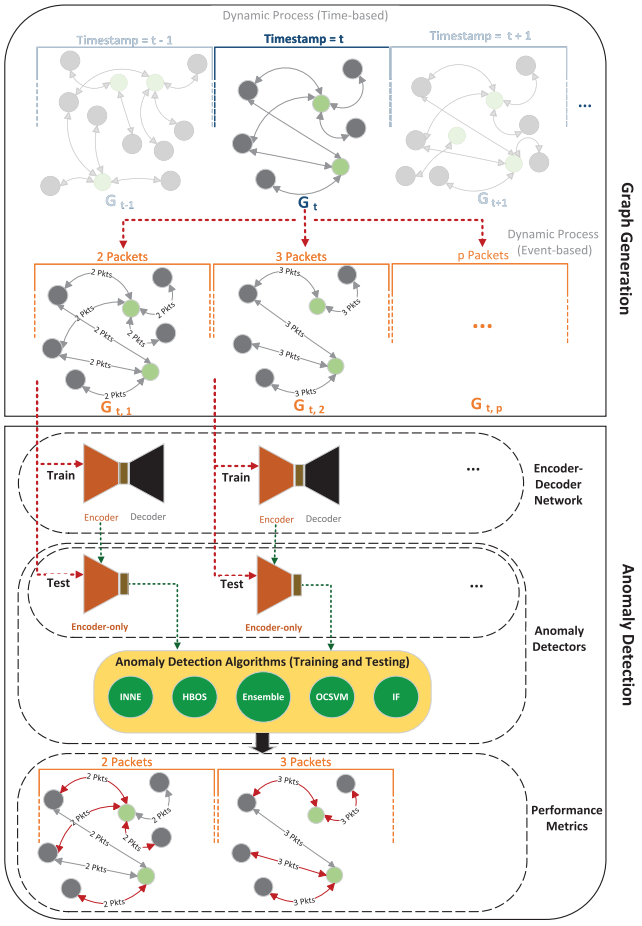


Fig. 1. GTAE-IDS Framework: The anomaly detection process is depicted exclusively for communication graphs involving 2 packets and 3 packets, denoted as $G_{t,2}$ and $G_{t,3}$, respectively. The red edges displayed at the conclusion of the process within the performance metrics section highlight the anomalous edges identified by the preceding anomaly detection algorithms.

arbitrary timestamp t. The graph structure changes with each event, depending on p in each evolving communication.

Our objective is to detect anomalous activities in the dynamic graph, represented as $G_{t,p} = (V_{t,p}, E_{t,p}, X_{v,t,p}, X_{e,t,p})$, where $G_{t,p}$ is the graph snapshot of the network at timestamp t and event p, comprising nodes $V_{t,p}$, edges $E_{t,p}$, and node and edge features $X_{v,t,p}$ and $X_{e,t,p}$, respectively. To achieve this objective, we propose a graph transformer-based autoencoder framework. A transformer neural network architecture with a self-attention mechanism can effectively capture intricate relationships and dependencies among nodes and edges. The encoder-decoder neural network architecture can analyze the graph data, learning a latent space that captures essential graph features, and can reconstruct the original graph data based on this learned representation. Figure 1 illustrates our graph transformer-based autoencoder intrusion detection system (GTAE-IDS) framework comprising two main components: (i) graph generation and (ii) anomaly detection. Next, we present the details of these components.

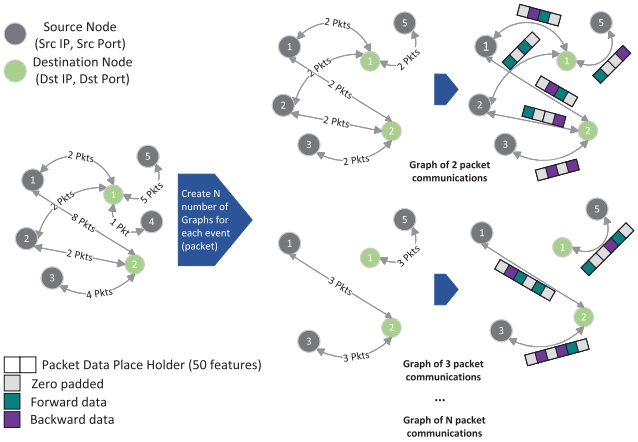


Fig. 2. Graph generation process (only the generation of graph networks for 2 and 3 packets is shown).

A. Graph Generation Component

The network traffic is continuously monitored, and relevant information is captured for any given timestamp t, which could be set by an organization in minutes or hours. This information encompasses packet data for each communication within the network traffic. The preferred format for storing network traffic data is the packet capture (*pcap*) format, widely acknowledged as the *de facto* standard for network packet capture. This format is widely used in packet sniffers and analysis tools like Wireshark [24]. Multiple open-source tools capable of performing this task exist, including well-known options such as libpcap [25] and tshark [26]. The goal of the graph generation component is to construct distinct graph representations ($G_{t,p} = (V_{t,p}, E_{t,p}, X_{v,t,p}, X_{e,t,p})$) based on the varying number of packets p within the network traffic at each timestamp t. To generate each graph, it is necessary to identify the characteristics of each $G_{t,p}$, including the nodes $V_{t,p}$ and their features $X_{v,t,p}$, edges $E_{t,p}$ and their features $X_{e,t,p}$, as well as the graph type (whether directed or undirected). Next, we discuss the details about the nodes, edges, and the graph type.

1) Nodes: In network communications, each flow or session is uniquely identified by a four-tuple from source (*Src*) and destination (*Dest*), given by ($SrcIP$, $SrcPort$, $DestIP$, $DestPort$). In the context of this study, to distinguish between nodes, we employ these identifiers to represent the nodes of the graphs. Consequently, $V_{t,p} = V_{t,p}^{SrcIP,SrcPort} \cup V_{t,p}^{DestIP,DestPort}$. Furthermore, we model the nodes to be devoid of any intrinsic attributes ($X_{v,t,p} = \emptyset$), capturing solely the global features of specific traffic data through a graph representation of interactions among source and destination addresses. Figure 2 shows the graph generation process. Notably, the graph on the left represents the snapshot of the network at a timestamp with p number of packets, with destinations depicted in green circles and sources in grey circles. The numbers on the edges indicate the quantity of packets transmitted by each flow or session at the time of the snapshot capture. Note that during the decomposition of these snapshots into different graph representations ($G_{t,p}$), some nodes may not be present in subsequent graphs.

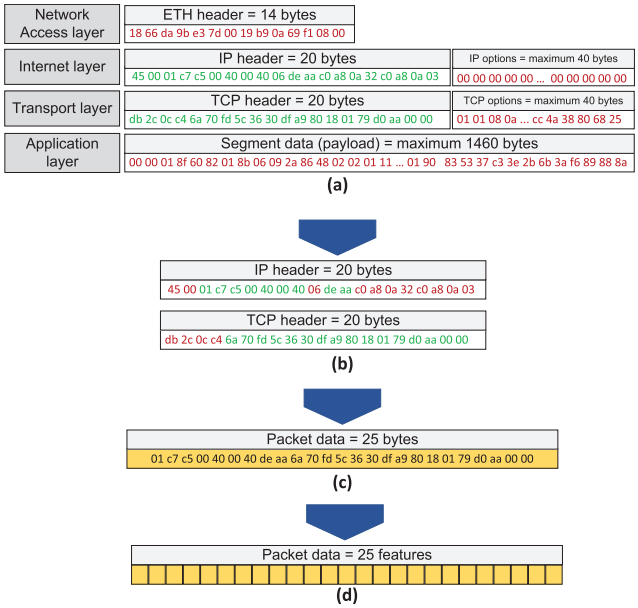


Fig. 3. Packet data structure.

2) Edges: For every pair of source and destination nodes, an edge is established if a flow or session exists between them. Additionally, the packet data transmitted through such flow or session is utilized for the edge features ($X_{e,t,p}$) of the graphs. Understanding the process of edge feature encoding employed in this study requires familiarity with the computer network standards and protocols. The Transmission Control Protocol/Internet Protocol (TCP/IP) model serves as the standard model in network communication, governing the process of information exchange across the internet. It comprises four layers: the network access layer (also known as the host-to-network layer), the internet layer, the transport layer, and the application layer.

Figure 3 (a) illustrates the distinct TCP/IP layers, accompanied by the number of information bytes at each layer. Each packet transmitted through the TCP protocol can contain up to 1594 information bytes. However, not every information byte contained within a packet is pertinent to the training process, and there may be limitations in accessing certain bytes. We outline the steps taken to prepare packet data to ensure the extraction of the most concise and relevant information conducive to efficient training.

- Information pertaining to the environment and protocols can introduce bias to the model, potentially limiting its applicability in diverse environments. To mitigate this bias, the Ethernet (ETH) header information (14 bytes), which includes the physical *MAC* address, is excluded from each packet. Furthermore, to prevent misalignment issues between two packets of the same flow and minimize noise in the model, both IP options and TCP options are eliminated. Misalignment can occur when the bytes in two feature representations of packets with and without options are not synchronized, resulting in a reduction in model performance and interpretability [27]. The segment data (payload), which may contain encrypted information in various protocols like *HTTPS, FTPS, etc.*, is also removed. Additionally, considering that many packets within a flow or connection lack payload data, the segment data is systematically removed from each packet. These information bytes, depicted in red in Figure 3 (a), are therefore removed for the aforementioned considerations.
- Information associated with the utilized IPs, Ports, and Protocols has the potential to introduce bias to the model, rendering it less applicable to diverse environments. Consequently, specific elements, including the IP version (one byte - 45), the differentiated services field (1 byte - 00), the protocol (one byte - 06), and the information pertaining to source and destination IP addresses (four bytes each - c0 a8 0a 32 & c0 a8 0a 03), are excluded from the IP header. Moreover, the information bytes concerning source and destination ports (two bytes each - db 2c & 0c c4) from the TCP header of each packet are also removed. These identified information bytes, highlighted in red in Figure 3 (b), undergo removal to address potential bias and enhance the model's adaptability across different environments.
- After the removal of these information bytes (totaling 1569 bytes), the resulting packet data consists of 25 bytes of information, as depicted in Figure 3 (c). Each byte signifies a feature in the packet-based feature representation. Subsequently, a byte-wise transformation is applied to the packet-based features, converting the hexadecimal byte values to their respective decimal values. The decimal values range from 0 (for 00 byte) to 255 (for *ff* byte). Finally, each feature is normalized to a range from 0 to 1, as illustrated in Figure 3 (d).

Next, this processed packet data undergoes encoding into the edge features of the constructed graph representations.

3) Graph Type: Network communication involves the back-and-forth exchange of data between a source node and a destination node. In this context, the packets transmitted from a source node to a destination node are termed "forward (fwd)" packets, while the reply packets from a destination node to a source node are referred to as "backward (bwd)" packets. Certain attacks, such as *SYN flooding*, aim to keep the destination (server) occupied and exhaust its resources by inundating it with multiple null packets devoid of malicious data [28]. Therefore, distinguishing between forward (attacker to server) and backward (server to attacker) packets becomes crucial. Consequently, encoding the direction of each packet in the edge features is necessary.

Although this study adopts an undirected graph structure, we propose a novel method to incorporate the directionality information into the edge features. The edge features encoding process considering the packets directionalities is illustrated in Figure 4. As mentioned earlier, each processed packet, regardless of direction, comprises 25 normalized features within the range of 0 to 1. To distinguish between fwd and bwd packets, we introduce a 50-feature long placeholder for each packet. For fwd packets, the first half (25 features) are populated with the packet data, and the second half of the packet placeholder is zero-padded. Conversely, for bwd

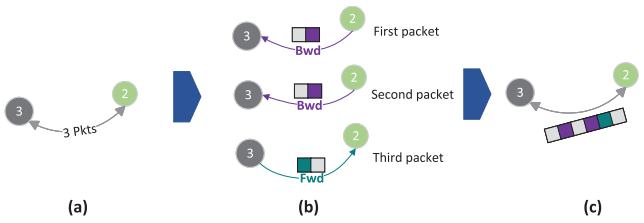


Fig. 4. The edge features encoding process.

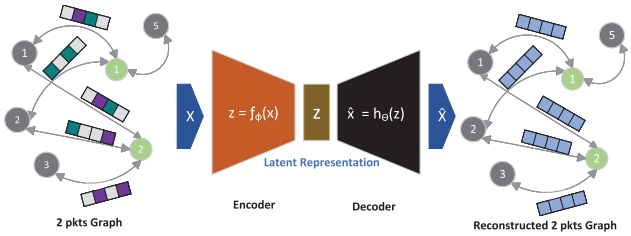


Fig. 5. The graph transformer-based autoencoder model (GTAE).

packets, indicating transmission from a destination node to a source node, the second half of the packet placeholder is filled with the packet data, while the first half is zero-padded.

The proposed graph generation procedure provides a logical and effective way to capture information within bidirectional flows into edge features due to several reasons:

- By allocating specific features to represent the direction of packets allows ML models to distinguish between packets transmitted from the source node to the destination node and packets transmitted from the destination node to the source node.
- This approach provides a clear and intuitive representation of packet direction within the edge features. By splitting the placeholder into two halves and populating each half differently based on the direction of the packet, a structured and easily interpretable encoding scheme can be created.
- The proposed method generates machine-readable features that are suitable for input into ML models. By incorporating directionality information directly into the feature space the model's ability to learn and recognize patterns associated with different directions of packet transmission can be enhanced.
- Allocating a fixed length placeholder for each packet ensures consistency in feature dimensionality, facilitating uniform processing across all packets. Additionally, the zero-padding of unused features ensures that the model can effectively differentiate between meaningful packet data and empty placeholders.

B. Anomaly Detection Component

In the complex landscape of network security, the effectiveness of classifiers within the supervised learning framework and anomaly detectors within the unsupervised learning framework stands out as a pivotal focus. Each detection method possesses distinct strengths. An anomaly detector excels in identifying deviations from historical network communication patterns, regardless of whether these anomalies are linked to known or new attacks. Therefore, we leverage an unsupervised learning approach to develop NIDS capable of uncovering emerging patterns in traffic, including those associated with previously unseen attacks. The autoencoder (AE) technique, a promising deep learning method, operates within the unsupervised learning paradigm by learning a compressed representation of input data samples without requiring explicit labels. AE models have been widely explored in literature for constructing NIDS using flow data [29], [30], [31]. These models consist of an encoder, tasked with decomposing the input features (x) into a lower-dimensional space, namely the bottleneck or latent representation (z), and a decoder network that reconstructs a representation ($\hat{x}$) of the input features from the latent representation [32].

Figure 5 illustrates an AE model consisting of an encoder function $f(\cdot)$ and a decoder function $h(\cdot)$, parameterized by ϕ and θ, respectively. The encoder network compresses the original high-dimensional input $\mathbf{x}$ into a low-dimensional code denoted by $\mathbf{z} = f_\phi(\mathbf{x})$ within the bottleneck layer. Subsequently, the decoder network reconstructs the input by generating $\hat{\mathbf{x}} = h_\theta\left(f_\phi(\mathbf{x})\right)$. Throughout the training phase, the parameters (θ, ϕ) are jointly learned to produce a reconstructed data sample closely resembling the original input, i.e., $h_\theta\left(f_\phi(\mathbf{x})\right) \approx \mathbf{x}$, effectively learning an identity function. A loss function quantifies the difference between the original input data (x) and the reconstructed output ($\hat{x}$). Various loss functions can be employed based on the use case and data characteristics. For instance, if the feature sets exhibit different scales, mean absolute error (MAE) could be a suitable loss function. However, in this study, the feature sets are of the same scale. Furthermore, our emphasis is on the precision of reconstruction, and we aim to penalize larger errors more significantly than smaller errors. Therefore, we employ mean squared error (MSE) as the loss function, which can be calculated for n number of samples as follows:

$$\mathcal{L}_{\mathrm{AE}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^{n} \left(\mathbf{x}^{(i)} - h_\theta\left(f_\phi\left(\mathbf{x}^{(i)}\right)\right)\right)^2 \tag{1}$$

In this study, we introduce an asymmetrical GTAE model incorporating graph transformer layers for constructing the encoder network and a deep neural network (DNN) architecture for the decoder network. Such a design enables the proposed GTAE to demonstrate the following capabilities:

- The global features of the graph for benign traffic are captured through the utilization of graph transformer layers in the encoder network.
- Employing a straightforward DNN model for the decoder facilitates the training process.
- After training the GTAE, the encoder model is detached and utilized in the subsequent phases, offering a lightweight network suitable for deployment.

1) Encoder Network: The graph transformer layers employed in the encoder network follow a methodology similar to the one proposed in [33], which supports edge features. In the case of a graph G with edge features $\gamma_{ij} \in \mathbb{R}^{d_e \times 1}$ for each edge between nodes i, j, the input edge features γ_{ij} undergo

a linear projection to embed them into d-dimensional hidden features e^0_{ij}.

$$e^0_{ij} = K^0 \gamma_{ij} + k^0, \tag{2}$$

where $K^0 \in \mathbb{R}^{d \times d_e}$ and $k^0 \in \mathbb{R}^d$ represent the parameters of the linear projection layers. However, as there are no node features available, we proceed to embed Laplacian positional encodings (λ_{ij}) [34] of dimension k through a linear projection, and subsequently, add them to the edge features, resulting in $\hat{e}^0_{i,j}$.

$$\lambda^0_{ij} = F^0 \lambda_{ij} + f^0 \quad ; \quad \hat{e}^0_{ij} = e^0_{ij} + \lambda^0_{ij}, \tag{3}$$

where $F^0 \in \mathbb{R}^{d \times k}$ and $f^0 \in \mathbb{R}^d$. It is important to note that the Laplacian positional encodings are exclusively added to the edge features at the input layer and not during intermediate layers. We now proceed to define the edge update equations for a layer ℓ.

$$\hat{e}^{\ell+1}_{i,j} = O^\ell_e \|_{k=1}^H \left(\sum_{j \in \mathcal{N}_i} w^{k,\ell}_{ij} V^{k,\ell} e^\ell_{i,j} \right), \tag{4}$$

where,

$$w^{k,\ell}_{ij} = \mathrm{SoftMax_j} \left(\frac{\mathrm{Q^{k,\ell}e^\ell_{i,j}} \cdot \mathrm{K^{k,\ell}e^\ell_{i,k}}}{\sqrt{d_k}} \right), \tag{5}$$

wherein, Q denotes the query matrix, K denotes the key matrix, V denotes the value matrix, and $Q^{k,\ell}, K^{k,\ell}, V^{k,\ell} \in \mathbb{R}^{d_k \times d}$, $O^\ell_e \in \mathbb{R}^{d \times d}$, and $k = 1$ to H represents the number of attention heads, with $\|$ denoting concatenation. The attention outputs $\hat{e}^{\ell+1}_{i,j}$ are subsequently forwarded to a FNN, followed by residual connections and normalization layers, as:

$$\hat{\hat{e}}^{\ell+1}_{ij} = \mathrm{LayerNorm} \left(\mathrm{e^\ell_{ij}} + \hat{\mathrm{e}}^{\ell+1}_{\mathrm{ij}} \right), \tag{6}$$

$$\hat{\hat{\hat{e}}}^{\ell+1}_{ij} = W^\ell_{e,2} \mathrm{ReLU} \left(\mathrm{W^\ell_{e,1}} \hat{\hat{\mathrm{e}}}^{\ell+1}_{\mathrm{ij}} \right), \tag{7}$$

$$e^{\ell+1}_{ij} = \mathrm{LayerNorm} \left(\hat{\hat{\mathrm{e}}}^{\ell+1}_{\mathrm{ij}} + \hat{\hat{\hat{\mathrm{e}}}}^{\ell+1}_{\mathrm{ij}} \right), \tag{8}$$

where $W^\ell_{e,1} \in \mathbb{R}^{2d \times d}$, $W^\ell_{e,2} \in \mathbb{R}^{d \times 2d}$, $\hat{e}ij^{\ell+1}$, and $\hat{\hat{e}}ij^{\ell+1}$ denote intermediate representations. The edge representations obtained at the final layer of the encoder model (z) serve as the bottleneck features, subsequently fed into a DNN-based decoder model to reconstruct the original edge features (x).

2) Decoder Network: The decoder is tasked with reconstructing the input features (x) from the compressed representation obtained in the bottleneck layer (z). Essentially, the decoder maps the encoded features back to the original input space. Let's denote the DNN layers as D_i, where i represents the layer index. The decoder can be expressed as:

$$z_{i+1} = W_i z_i + b_i, \tag{9}$$

$$a_{i+1} = g\left(z_{i+1}\right), \tag{10}$$

where W_i is the weight matrix, b_i is the bias vector, z_i is the input to layer D_i, z_{i+1} is the output after the linear transformation, $g(\cdot)$ is the activation function, and a_{i+1} is the output after activation. The output layer is denoted by:

$$\hat{x} = W_{\text{out}}\ z_{\text{final}}\ + b_{\text{out}}\ , \tag{11}$$

where W_{out} and b_{out} are the weights and biases of the final output layer, and z_{final} is the output of the last layer.

The specifics of the GTAE model, including the choice of activation functions, the number of graph transformer layers (L), the number of attention heads (H), the number of DNN layers (D), and neurons in each layer, may vary depending on the input graph representation. We describe the specifics of the model in Section IV.

3) Anomaly Detectors: Next, we discuss the generation of edge embeddings for the train and test graphs using the encoder network of the trained GTAEs (encoder-only), followed by the edge embedding results of the training graphs employed as part of the training data for the anomaly detection algorithms. Subsequently, we utilize the resulting edge embeddings from the test graphs in the anomaly detection algorithms to identify anomalous edges. In the anomaly detection algorithms, we use ML models instead of traditional thresholding that uses anomaly scores statistics such as mean and standard deviation. The reason is that in the thresholding mechanism, the anomaly score is determined by human experts and often finding the optimal thresholding value is time-consuming and prone to error. However, ML-based anomaly detection algorithms use benign traffic data for training in a fully unsupervised approach, aiming to identify instances that differ significantly from the benign traffic distribution. By training on a data set composed of benign data embeddings, these algorithms can establish a reliable profile of normal behavior, which then helps to identify deviations that may signify anomalies. During testing, both normal and anomalous data points (embeddings) are evaluated. The models output a value of 0 for normal activities and -1 for anomalies: a 0 indicates that the test instance aligns with the benign data profile, while a -1 signals a significant deviation, aligning with malicious activity.

In this study, we adopted a multi-faceted approach to comprehensively analyze available anomaly detection algorithms. Our approach incorporates candidates from different perspectives, including proximity-based, outlier ensembles, and linear model approaches. The proximity-based approach leverages the distribution of data to compute outlier scores efficiently. It focuses on analyzing the proximity of data points within a feature space to identify anomalies effectively. The ensemble approach aims to identify anomalies by isolating them into smaller partitions within a random forest structure. By aggregating the results of multiple base learners, it enhances the robustness and accuracy of anomaly detection. Lastly, the linear model approach relies on constructing a hyperplane to segregate normal instances from anomalies within the feature space. It utilizes linear separation techniques to classify data points as either normal or anomalous based on their position relative to the hyperplane.

In particular, we analyze several anomaly detection algorithms from the aforementioned three approaches: (i) the histogram-based outlier score (HBOS) [35] algorithm (proximity-based), (ii) the isolation forest (IF) [36] and the isolation-based anomaly detection using nearest-neighbor ensembles (INNE) [37] (outlier ensembles), and (iii) the one-class support vector machines (OCSVM) algorithm [38] (linear model). We not only utilize these individual anomaly detection algorithms but also integrate their detection out-

comes in a voting-based ensemble method to enhance robustness and accuracy beyond the capabilities of individual models. Specifically, we introduce a majority voting ensemble classifier where each algorithm "votes" for a specific class label (either benign or malicious). The final prediction is determined by selecting the class label that receives the most votes. This ensemble approach aims to leverage the diverse strengths of individual algorithms, contributing to an improved and more reliable anomaly detection system. Below we provide a brief description of each of these algorithms.

- *One-class support vector machines (OCSVM):* The OCSVM is a linear approach that forms a hyperplane to encapsulate normal instances while minimizing the influence of anomalies. OCSVM is effective in high-dimensional spaces and is particularly useful when normal instances occupy a small fraction of the feature space. The objective function is defined as $\min_{w,\xi,\rho} \frac{1}{2}w^T w + \frac{1}{\nu}\sum_{i=1}^{m}\xi_i - \rho$, subject to $\langle w, \phi(x_i)\rangle \geq \rho - \xi_i$ for normal instances and $\langle w, \phi(x_i)\rangle \leq \rho + \xi_i$ for anomalies, where, w is the weight vector of the hyperplane used to separate normal instances from anomalies, ξ is slack variable introduced to allow for some misclassification of data points, ρ denotes the offset or bias term, ν is a parameter that controls the trade-off between maximizing the margin and minimizing the number of outliers, $\phi(x_i)$ denotes the feature mapping function, and m represents the number of training instances.
- *Isolation forest (IF):* The IF technique, being an ensemble method, excels at isolating anomalies with ease, often requiring fewer partitions to highlight them prominently. IF demonstrates efficiency, especially in scenarios involving high-dimensional data and rare anomalies. The isolation of an instance x is defined as $h(x) = E(h(x))$, where $h(x)$ is the path length to reach x in the tree, and $E(h(x))$ is the expected path length for an average instance.
- *Histogram-based outlier score (HBOS):* The HBOS method utilizes histograms to estimate the likelihood of observing data points within each feature. HBOS is computationally efficient, particularly suited for large data sets, and less sensitive to parameter tuning. The outlier score for an instance x is calculated as $\text{Score}(x) = \prod_{i=1}^{d} P_i(x_i)$, where $P_i(x_i)$ is the probability of observing x_i in the histogram of the i-th feature and d denotes the dimensionality of the data, i.e., the number of features.
- *Isolation-based anomaly detection using nearest neighbor ensembles (INNE):* The INNE technique is also an ensemble algorithm which combines isolation with a nearest-neighbor approach, emphasizing the isolation of instances based on their distance from their nearest neighbors in a nearest-neighbor ensemble. INNE enhances anomaly detection by considering the local density of instances and their relationships with neighbors. Similar to IF, the isolation of an instance x is defined as $h(x) = E(h(x))$, where $E(h(x))$ is the expected path length for an average instance.

4) Evaluation Metrics: The evaluation of the anomaly detectors' performance in the GTAE framework is conducted using two important metrics commonly employed for intrusion detection models, namely, true positive rate (TPR) and false positive rate (FPR). These metrics are calculated from the testing graph samples, as follows:

- TPR: It is also known as the detection rate (DR) and is the ratio of correctly detected anomalous samples (TP) and the total number of anomalies.
- FPR: It is also known as the false alarm rate (FAR) and is the ratio of incorrectly detected benign samples as anomalies (FP) and the total number of benign samples.

IV. Numerical Experiment Setup

This section provides a comprehensive overview of the numerical experiments undertaken to evaluate our methodology. First, we elaborate on the network traffic data employed in the experiments. Next, we outline the process of generating graph sets for training and testing purposes. Finally, we delve into the specifics of the models and the parameters utilized within the framework.

Our experiments were performed utilizing a 12th Generation Intel Core i9-12950HX processor, featuring a 30 MB cache, 24 threads, and 16 cores. To enhance the training efficiency, an NVIDIA RTX A5500 graphics card with 16 GB GDDR6 SDRAM was employed, along with the latest installations of CUDA, a universal parallel computing framework, and cuDNN, a deep neural network acceleration library.

A. Data Description

We conducted numerical experiments utilizing three widely used publicly available data sets: CIC-IDS2017 [12], CIC-IDS2018 [13], and ACI-IoT-2023 [14]. Notably, in comparison to other publicly accessible data sets such as NSL-KDD [39] and KDD-CUP [40], these data sets are more recent, offering more realistic representations of contemporary network traffic [41]. These data sets comprise raw traffic data capturing various attack and benign communications in pcap files distributed across specific days of the week. The pcap files contain raw network packet data; however, these packets lack labels, in contrast to readily available flow data sets. We processed these pcap files following the procedures outlined in [42], [43] to extract packet data and label them based on the metadata provided by the Canadian Institute for Cybersecurity (CIC) [44] and the Army Cyber Institute (ACI) [45].

B. Train and Test Graphs

While the CIC-IDS2017, CIC-IDS2018, and ACI-IoT-2023 data sets are created over a span of multiple days, in the context of our framework, we consider them as a snapshot of network traffic at an arbitrary timestamp t. To showcase the effectiveness of our approach, we generate multiple sample graph sets at different stages of the framework, including the training of the GTAEs, training the anomaly detection algorithms, and testing. Specifically, the benign data is employed in three key aspects of the framework: training graph sets for the GTAEs (train sets 1), training graph sets for the anomaly detection algorithms (train sets 2), and combining benign

TABLE II

CHARACTERISTICS OF THE TRAINING AND TESTING GRAPH SETS

p (# Packets)	# Edge Features	Graph Size (# Nodes, # Edges) CIC-IDS2017 Train 1 (Only Benign)	CIC-IDS2017 Train 2 (Only Benign	CIC-IDS2017 Test (Benign and Malicious)	CIC-IDS2018 Test (Only Malicious)
2	100	(7160, 5000)	(3736, 2000)	(44767, 38984)	(10792, 10443)
3	150	(7144, 5000)	(3351, 2000)	(46285, 37971)	(7439, 7166)
4	200	(7135, 5000)	(3145, 2000)	(45008, 36964)	(6833, 6669)
5	250	(7135, 5000)	(3143, 2000)	(44118, 36093)	(6569, 6418)
6	300	(7129, 5000)	(3136, 2000)	(43417, 35482)	(6332, 6197)
7	350	(7112, 5000)	(3126, 2000)	(41643, 33974)	(6176, 6064)
8	400	(7023, 5000)	(3106, 2000)	(40744, 33228)	(6116, 6012)

TABLE III

SPECIFICATION DETAILS OF VARIOUS GTAE MODELS

GTAE (p)	Encoder # Layers	# Heads	# In Features	# Out Features	Decoder # Layers	# Neurons/Layer	Activation
GTAE (2)	3	3	100	8	2	32, 64	ReLU
GTAE (3)	3	3	150	8	2	32, 64	ReLU
GTAE (4)	4	4	200	16	3	32, 64, 128	ReLU
GTAE (5)	5	6	250	32	3	32, 64, 128	ReLU
GTAE (6)	5	6	300	32	3	32, 64, 128	ReLU
GTAE (7)	6	8	350	64	4	32, 64, 128, 256	ReLU
GTAE (8)	6	8	400	64	4	32, 64, 128, 256	ReLU

TABLE IV

HYPERPARAMETER VALUES USED FOR ALL GTAE MODELS

Hyperparameters	Best Value
Learning Rate	0.001
# Epochs	100
Batch Size (# Subgraphs)	50
Optimizer	Adam
Attention Dropout (Last Layer)	0.1
FFN Dropout (Each Layer)	0.05

TABLE V

HYPERPARAMETER SELECTION AND VALUES USED IN GRID SEARCH FOR ANOMALY DETECTION ALGORITHMS

Algorithm	Hyperparameters	Values
OCSVM	Kernel	['linear', 'poly', **'rbf'**, 'sigmoid']
IF	# Estimators	[20, 50, 100, **150**]
INNE	# Estimators	[50, **100**, 150, 200]
HBOS	# Bins	[5, 10, 15, **20**, 25, 30]
All	Contamination	[0.001, **0.01**, 0.04, 0.05, 0.1]

and anomaly data to create testing graph sets for framework evaluation. When creating the graph sets, we take into account the following criteria:

- We exclusively utilize benign samples from the CIC-IDS2017 data set for training both the GTAEs and anomaly detection algorithms. The CIC-IDS2018 and ACI-IoT-2023 data sets are then reserved for additional framework evaluation, demonstrating its effectiveness and generalization in detecting novel attacks or anomalies within a new environment.
- As outlined in the graph generation process, each flow corresponds to an undirected edge in the graph. The quantity of benign data utilized to construct the training graph sets 1 & 2 and testing graph sets for various graph representations p is consistent and comprises 5000, 2000, and 5000 edges, respectively.
- In the testing phase, we incorporate anomaly data from each attack type. Given the substantial volume of anomalous data for the *DDoS* and *DoS Hulk* classes, we limit the usage to 5000 edges for each of these attack types.
- The number of edge features for different $G_{t,p}$'s is equal to $2 \times 25 \times p$.
- The initial packet in any communication/flow is typically a synchronize (*SYN*) packet, devoid of any malicious intent. Consequently, we initiate the creation of graph sets from $p = 2$ packets. Furthermore, we continue generating graph sets up to $p = 8$ packets, as indicated in [43], where this number of packets per communication proves sufficient for anomaly detection.

The characteristics of the sample graph sets utilized to generate various graphs during both the training and testing phases are detailed in Table II. For example, in the case of $p = 3$, indicating a snapshot of dynamic graphs for flows comprising three packets each, the edge features amount to 150 features. Furthermore, the training data set 1 comprises a graph encompassing benign flows, consisting of 7144 nodes (including source and destination IPs/Ports) and 5000 edges representing the flows.

C. Model Specifics and Parameters

For each graph representation, the training of the GTAE model involved subgraph sampling from the corresponding training graph set (train 1). Each GTAE model comprises a transformer-based encoder and a DNN-based decoder, as discussed earlier in Section III. The number of graph transformer and DNN layers in both encoder and decoder parts varies depending on the graph representation, as outlined in Table III. The remaining hyperparameters are selected based on best empirical values through experimentation using grid search, as shown in Table IV.

After completing the training of GTAEs, the optimized encoder-only models were employed to generate edge embeddings (i.e., bottleneck features) from both the training graphs (train 2) and testing graphs for each graph representation. The train 2 graph sets consist solely of benign traffic data, as outlined in Table II. These train 2 graphs are used with the encoder-only model to generate edge embeddings, which are then applied to train the ML-based anomaly detection models. Importantly, a distinct set of benign data is used to construct the graphs at this stage, ensuring no overlap with the benign data used in the train 1 graph sets that trained the GTAE. This approach allows for a more realistic and robust profiling of the benign data by training the ML-based anomaly detection algorithms on a different set of benign graphs. In the evaluation phase, both benign and anomalous data points are included to create graph sets from the CIC-IDS2017, CIC-IDS2018, and ACI-IoT-2023 data sets, as previously noted. We utilized four algorithms: OCSVM, IF, INNE, and HBOS for anomaly detection.

We employed the grid search method in each experiment for parameter tuning, ensuring optimal configurations for each detection algorithm. The grid search involved parameters such as contamination for all algorithms, the kernel type for OCSVM, the number of estimators for IF, the number of estimators for INNE, and the number of bins for HBOS. Table V provides a detailed overview of the grid search parameters. The contamination parameter plays a critical role

TABLE VI
MODEL PERFORMANCE ON CIC-IDS2017 DATA SET ACROSS DIFFERENT GRAPH REPRESENTATIONS

Graph Representation	Graph Size (Nodes, Edges)	Anomaly Detection Methods									
		OCSVM		IF		HBOS		INNE		Ensemble (Voting-based)	
		TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR
2 Packets	(44767, 38984)	81.56	3.4	79.78	5.19	77.8	5.16	83.75	4.83	84.47	8.94
3 Packets	(46285, 37971)	90.47	3.81	90.6	4.83	90.58	4.78	90.8	5.04	91.02	7.61
4 Packets	(45008, 36964)	94.39	3.98	94.36	4.96	94.52	4.87	94.46	4.89	94.72	6.6
5 Packets	(44118, 36093)	96.48	3.79	95.36	4.83	95.7	4.84	95.08	5.13	97.16	6.84
6 Packets	(43417, 35482)	98.02	3.7	96.01	5.1	96.75	5.36	97.32	5.13	98.44	6.73
7 Packets	(41643, 33974)	98.12	3.51	95.14	4.76	97.62	4.75	98.21	4.88	98.72	5.91
8 Packets	(40744, 33228)	98.46	4.03	96.66	4.93	97.73	4.96	96.47	4.99	99.04	6.56

in setting the expected proportion of anomalies within the data set. In the grid search, we evaluated values ranging from 0.001 to 0.1 to assess how sensitive the models were to different contamination levels. Lower contamination values (e.g., 0.001 and 0.01) were tested to determine the models' robustness in cases with a very small fraction of anomalies, while higher values (e.g., 0.05 and 0.1) were explored for data sets with a moderate level of anomalies. We observed that as the contamination level increased, detection rates generally improved due to the model's increased focus on potential anomalies. However, excessive contamination values led to higher false positives, as the models began labeling more normal traffic as anomalous. The optimal contamination values, as indicated by performance metrics, typically fell between 0.01 and 0.05, balancing high detection rates with lower false-positive rates. For OCSVM, the 'rbf' kernel showed superior anomaly detection due to its ability to capture complex patterns, while simpler kernels like 'linear' and 'poly' offered faster but less accurate results. In IF and INNE, increasing the number of estimators improved detection reliability, although values above 150 provided minimal additional benefits while raising computational costs. For HBOS, a moderate bin count (around 15-20) balanced granularity and detection accuracy, as too few bins missed subtle patterns, and too many led to overfitting. In Table V, the best hyperparameter values are highlighted in bold font after employing the grid search method. After training each algorithm, the test graphs from CIC-IDS2017, CIC-IDS2018, and ACI-IoT-2023 data sets were utilized to evaluate the model's success in an unsupervised manner, as illustrated in the 'Anomaly Detection' component in Figure 1.

V. ANALYSIS OF RESULTS

This section presents the results and analysis of the experiments conducted using our GTAE-IDS framework. We first demonstrate the efficacy of our approach using the test graph sets from CIC-IDS2017. We analyze the performance across the different graph representations and extract insights into the sufficient number of packets being transmitted in order that GTAE-IDS framework can detect anomalies with optimal precision. Next, we assess the GTAE-IDS framework performance in detecting novel anomalies from a new target environment, CIC-IDS2018. We then perform an ablation study to showcase the importance of the context and information provided by the GRL paradigm utilized in this study. Lastly, we conduct comparative analysis with the state-of-the-art approaches to assess the superiority of GTAE-IDS.

A. Performance Results on Cic-IDS2017 Test Graph Sets

Table VI shows the performance of the anomaly detection models on the CIC-IDS2017 test graph sets through the TPR and FPR metrics defined in Section III. It can be observed that the performance improves as the dynamic graph evolves in terms of the total number of packets in the flows, as evidenced by the metric values. The best performance across the metrics is observed with graph representation of eight packets with an anomaly detection rate (TPR) of 99.04% for the voting-based ensemble model. Achieving such a high TPR when only the first few packets are communicated in a flow indicates that formulating network traffic as a dynamic graph representation using GTAE-IDS enables quick and accurate detection of anomalies in network traffic.

Next, we present the performance of the model in detecting anomalies across different types of network attacks. The CIC-IDS2017 testing graph sets were generated to include flows (edges) from all attack classes in the graph representation along with benign edges. In the previous experiment we have treated all attack classes edges as anomalous edges to evaluate the GTAE-IDS performance. However, it is equally important to see how our framework is performing when exposed to malicious traffic associated with specific attack types. For this experiment, we extracted new testing graph sets with each set containing edges only belonging to a specific attack type. We generated graph sets for all attack types presented in the CIC-IDS2017 pcap file. Table VII shows the anomaly detection results per attack type using CIC-IDS2017 test graph sets for the eight packets graph representation. The anomaly detection rates across all attack types exceed 98.5%, indicating that GTAE-IDS is able to identify the malicious traffic representing all network attacks with high accuracy.

B. Performance Results on Cic-IDS2018 and Aci-IoT-2023 Test Graph Sets

Next, we assess the GTAE-IDS performance and generalization in detecting anomalies in new environments. In this

TABLE VII
ANOMALY DETECTION RESULTS PER ATTACK TYPE USING CIC-IDS2017 TEST GRAPH SETS FOR 8-PACKET GRAPH REPRESENTATION

Attack Type	Graph Size (Nodes, Edges)	OCSVM	IF	HBOS	INNE	Ensemble
DoS GoldenEye	(6235, 6090)	98.49	96.86	97.57	96.45	99.08
Heartbleed	(2032, 1468)	99.32	97.21	98.23	96.59	99.52
DoS Slowhttptest	(1248, 964)	99.59	98.34	99.17	98.55	99.79
DoS Slowloris	(1294, 1002)	99.20	97.80	98.60	97.21	99.60
DoS Hulk	(5382, 5000)	98.02	96.26	97.18	96.24	98.74
PortScan	(8242, 6451)	97.75	95.27	96.96	95.75	98.50
DDoS	(6072, 5000)	98.74	97.30	98.42	96.66	99.34
Infiltration	(2437, 1764)	99.66	98.36	98.81	97.73	99.83
Web Attack-BruteForce	(189, 140)	98.57	98.57	99.29	95.71	99.29
SSH-Patator	(248, 175)	98.86	97.14	98.29	96.00	98.86
FTP-Patator	(265, 174)	98.28	94.25	96.55	97.13	98.85

TABLE VIII
GTAE-IDS PERFORMANCE RESULTS ON CIC-IDS2018 AND ACI-IOT-2023 TEST GRAPH SETS ACROSS DIFFERENT GRAPH REPRESENTATIONS

Data Set	Graph Representation of	Graph Size (Nodes, Edges)	Ensemble
CIC-IDS2018	2 Packets	(10792, 10443)	81.63
	3 Packets	(7439, 7166)	91.27
	4 Packets	(6833, 6669)	93.67
	5 Packets	(6569, 6418)	97.05
	6 Packets	(6332, 6197)	98.51
	7 Packets	(6176, 6064)	98.65
	8 Packets	(6116, 6012)	98.74
ACI-IoT-2023	2 Packets	(2365, 2344)	79.02
	3 Packets	(1734, 1720)	88.94
	4 Packets	(1607, 1554)	93.21
	5 Packets	(1545, 1521)	94.87
	6 Packets	(1436, 1392)	96.76
	7 Packets	(1378, 1283)	97.88
	8 Packets	(1305, 1149)	98.21

experiment, the testing graph sets only include the malicious traffic data from the CIC-IDS2018 and ACI-IoT-2023 data sets. Table VIII presents the GTAE-IDS anomaly detection results for graph representations of two to eights packets for the best method (ensemble model). The high detection results, over 98%, for the eight-packet graph representations from CIC-IDS2018 and ACI-IoT-2023 test graphs further validate the effectiveness of GTAE-IDS in detecting anomalies irrespective of the network environment. We further show the anomaly detection results per attack type for graph representation of eight packets for both CIC-IDS2018 and ACI-IoT-2023 data sets. Table IX shows the anomaly detection results per attack type for various graph representation up to eight packets for both CIC-IDS2018 and ACI-IoT-2023 data sets. From the 12 attack types in the ACI-IoT-2023 data set, we have selected six, *Port Scan, DoS Slowloris, ICMP Flooding, Ping Sweep, ARP Spoofing,* and *BruteForce*, as representative cases to demonstrate the generalization of our approach to a new network environment.

C. Ablation Study

We conducted an ablation study to systematically evaluate the impact of the GRL component on the overall performance of the GTAE-IDS framework. Specifically, we examine how providing context through graph representation enhances the ability of our framework to detect anomalies. GTAE-IDS leverages a graph transformer-based encoder network within an autoencoder architecture to capture the contextual and global features from benign graph data. Therefore, we aim to discern the impact of omitting graph representation by directly applying the anomaly detection algorithms to the edge features of graph sets. In other words, the edge features of the testing graphs are stored in a tabular format and then the anomaly detection algorithms are applied. Table X displays the performance of GTAE-IDS in detecting anomalies in data samples containing eight packets from CIC-IDS2017, covering both benign and malicious data. The differences in TPR and FPR between the approaches, with and without context (GRL), are presented in the last row of the table. There is an average improvement of 4.4% in TPR and 3.97% in FPR with GRL enabled. We further show the comparison of anomaly detection performance across individual attack types between both these approaches in the supplemental appendix (see Table XVI in the supplemental material).

D. Comparative Analysis

This section presents a comprehensive comparative analysis of our framework with other state-of-the-art methods within both the context of GNN-based methodologies and broader approaches utilized in the field of network intrusion detection. Firstly, we conduct a performance evaluation of GTAE-IDS in comparison to recent GNN-based methods. Secondly, we compare the results of our approach with other non-GNN methods in the literature that utilize packet information in their proposed NIDS. Lastly, we provide a detailed comparison of our framework with another contemporary method employing a similar sequential packet representation approach but without incorporating graph representation learning technique.

1) Comparison With GNN-Based Methods: Given the absence of existing GNN-based approaches employing sequential packets in an unsupervised context, we have assessed the performance of GTAE-IDS in comparison to recent state-of-the-art methods utilizing flow data for intrusion detection in

TABLE IX
ANOMALY DETECTION RESULTS PER ATTACK TYPE USING CIC-IDS2018 AND ACI-IOT-2023 GRAPH SETS FOR 8 PACKET GRAPH REPRESENTATION

Data Set	Attack Type	Anomaly Detection Rate (Ensemble)						
		2 Packets	3 Packets	4 Packets	5 Packets	6 Packets	7 Packets	8 Packets
CIC-IDS2018	DoS GoldenEye	76.6	84.04	90.21	94.58	97.08	97.39	99.5
	DoS Slowloris	89.48	92.4	95.44	97.47	99.37	99.15	99.47
	DDoS	100	99.5	93.75	98.9	99.4	99.8	98.67
	Infiltration	96	100	95.92	97.78	100	100	100
	Web Attack-BruteForce	86.18	93.93	98.37	99.23	99.71	99.32	100
	Web Attack XSS	90.16	95.06	97.24	96.72	99.2	97.41	99.39
	SSH-Patator	76.18	82.08	94.38	96.48	97.65	97.92	98.51
	FTP-Patator	64.58	92.39	100	100	100	100	100
ACI-IoT-2023	Port Scan	87.32	93.5	96.75	98.45	98.43	98.83	99.67
	DoS Slowloris	93.22	94.76	95.92	97.58	98.65	99.32	99.56
	ICMP Flooding	83.17	87.83	88.38	89.53	89.86	94.32	96.54
	Ping Sweep	91.15	96.6	97.34	96.79	98.45	98.41	99.19
	ARP Spoofing	76.18	84.18	84.38	86.98	93.65	95.82	97.31
	BruteForce	67.58	84.38	94.1	95.32	97.32	98.23	98.43

TABLE X
ANOMALY DETECTION RESULTS WITH AND WITHOUT GRAPH REPRESENTATION LEARNING (ABLATION ANALYSIS FOR 8-PACKET DATA)

Algorithms	**OCSVM**	**IF**	**HBOS**	**INNE**	**Ensemble**
Metrics	**TPR**	**TPR**	**TPR**	**TPR**	**TPR**
Without Context	95.81	88.45	90.01	95.34	96.75
With Context (GRL)	**98.46**	**96.66**	**97.73**	**96.47**	**99.04**
Difference (+/-)	**+2.65**	**+8.21**	**+7.72**	**+1.13**	**+2.29**

supervised settings. To accomplish this, we have chosen three methodologies from literature: E-GraphSAGE [7], GraphDDoS [22], and another recent GNN-based approach proposed in [15]. Experiments in this section are carried out using the CIC-IDS2017 data set. Each of these methods is evaluated in a binary manner, with a balanced distribution of benign and attack samples for each attack type. Table XI displays the detection results (TPR) for different attacks across all the approaches. The GTAE-IDS method demonstrated superior performance over the other algorithms for the seven attack types and achieved nearly comparable results for the remaining four attack types. Particularly noteworthy was the significant improvement observed for the *WebAttack-BruteForce, Infiltration,* and *DoS Slowloris* attack types, in comparison to other approaches. This suggests that the inclusion of anomalous data from packets aids in accurately identifying attacks. Conversely, when combined and aggregated to derive flow-based data, these anomalous patterns in data may be lost, potentially impacting the effectiveness of the detection process.

2) Comparison With Non-GNN Methods: We conducted a comparative analysis of our GTAE-IDS framework against several non-GNN NIDS that leverage packet-level information, as presented in recent literature. Table XII summarizes this performance comparison, showcasing detection rates and inference times for our approach alongside other DL models that were evaluated on the CIC-IDS2017 data set, including AEIDS [46], HAST-II [47], PL-RNN [48], Packet2Vec [49], PayloadEmbeddings [50], and PBCNN [51]. Key metrics such as detection rate (TPR) and inference time per packet (measured in microseconds, μs) are reported in Table XII. In terms of detection rate, GTAE-IDS outperforms all listed methods, achieving a TPR of 0.990, which surpasses even the best-performing non-GNN model, PBCNN, at 0.983. Additionally, our framework demonstrates a highly efficient inference time of 1.28 μs per packet, which is the lowest among all evaluated methods. This includes both the time required for graph generation and model inference. The results highlight the effectiveness of our approach in providing a rapid response with superior accuracy, making it promising for near real-time anomaly detection.

3) Comparison With Other Sequential Packet-Based Method: Our approach considers packet-level data and analyzes the temporal characteristics associated with the ongoing sequence of packets in a flow. To ensure a fair comparative analysis, we benchmark our approach against a recent method from the literature called sequential packets image-based network intrusion detection system (SPIN-IDS) [43]. SPIN-IDS utilizes packet-level data to construct images from the sequential packets within ongoing network communications. These images are then processed through a CNN-based classifier to detect malicious activities. Table XIII provides a comparison of our approach with SPIN-IDS across various representations, ranging from two to eight packets, using the CIC-IDS2017 data set. The results demonstrate the superior performance of our approach across all packet representations compared to SPIN-IDS. These results suggest that GTAE-IDS with the GRL paradigm effectively captures the structural and contextual information inherent in network traffic data, leading to enhanced attack detection accuracy. We further compare both their performances in detecting individual attack classes in the supplemental appendix, demonstrating the superior attack detection ability of GTAE-IDS. The results are shown in Table XVII in the supplemental material.

E. Efficiency Evaluation

Our methodological design enables the detection of anomalies without waiting for the completion of an entire network flow, which can sometimes range in hundreds of packets and

TABLE XI
COMPARISON RESULTS WITH THE STATE-OF-THE-ART GNN-BASED METHODS ON THE CIC-IDS2017 DATA SET

Method	Attack Class										
	DoS Slowloris	DoS Slowhttptest	DoS Hulk	DoS GoldenEye	Heartbleed	FTP-Patator	SSH-Patator	WebAttack-BruteForce	Infiltration	PortScan	DDoS
E-GraphSAGE	0.973	0.983	0.989	**0.992**	0.983	0.966	0.962	0.954	0.963	0.974	0.965
GNN-BASED	0.982	0.974	**0.993**	0.991	0.978	**0.993**	0.986	0.734	0.983	0.981	0.991
GraphDDoS	0.967	0.956	**0.993**	0.978	0.981	0.983	0.976	0.967	0.974	**0.991**	0.983
GTAE-IDS	**0.996**	**0.998**	0.988	0.991	**0.996**	0.989	**0.988**	**0.993**	**0.998**	0.985	**0.994**

TABLE XII
COMPARISON OF GTAE-IDS WITH NON-GNN STATE-OF-THE-ART NIDS FROM LITERATURE

Methods	Performance Metrics	
	Detection Rate (TPR)	Inference Time (μs)
PL-RNN	0.564	2.91
Packet2Vec	0.728	3.54
HAST-II	0.690	4.2
AEIDS	0.587	2.65
PayloadEmbeddings	0.954	2.73
PBCNN	0.983	7.89
GTAE-IDS	**0.990**	**1.28**

TABLE XIII
GTAE-IDS PERFORMANCE COMPARISON WITH SPIN-IDS FOR VARIOUS NUMBER OF PACKETS

Number of packets	TPR	
	SPIN-IDS	GTAE-IDS
2	81.59	**84.47**
3	86.50	**91.02**
4	92.02	**94.72**
5	94.47	**97.16**
6	97.54	**98.44**
7	97.54	**98.72**
8	98.77	**99.04**

cause significant delays. We construct graphs based on packet representations of up to eight packets, as literature (e.g., SPIN-IDS) and our findings suggest that malicious behavior can very often be identified early in the packet sequence. This allows us to detect anomalies promptly and minimize processing time by halting further graph construction once an anomaly is detected. For example, if an anomaly is identified at the 3-packet stage, we avoid processing up to eight packets, conserving resources and improving overall efficiency. GTAE-IDS operates with a time complexity of $O(c)$, where c represents the number of valid network communications. Each communication is represented by a single edge in the constructed graph, facilitated through dynamic packet processing, which enables our system to scale efficiently as network traffic increases. This design supports rapid anomaly detection without unnecessary computation, which is crucial in high-traffic environments where quick response times are essential.

Tables XV and XIV display the training time (in hours, h) and testing time (in microseconds, μs) of the various GNN-based methods and GTAE-IDS on the CIC-IDS2017 data set. The training time and the mean detection time

TABLE XIV
TRAINING AND TESTING TIMES OF GTAE-IDS FOR DIFFERENT STAGES OF THE FRAMEWORK FOR THE GRAPH SNAPSHOT OF $p = 8$

Stage	GCT	GRLT	ADT	Total
Training (h)	0.188	4.202	1.04	5.43
MDT (μs)	0.06	0.81	0.41	1.28

TABLE XV
TRAINING AND TESTING TIMES OF GNN-BASED METHODS

Method	Training (h)	MDT (μs)
E-GraphSAGE [7]	3.1	1.2
GNN-based [15]	2.7	1.45
GraphDDoS [22]	4.6	2.12

(MDT) presented in Table XV exclude the extraction time of initial features, specifically the statistical flow-based feature extraction time. Moreover, the time spent waiting for the completion of flows to extract the statistical features for the methods from literature is not considered for a fair comparison. In contrast, for GTAE-IDS we considered all the required stages of our framework from feature extraction and graph generation to anomaly detection process in Table XIV where GCT denotes the graph construction time, GRLT represents the graph representation learning time, and ADT indicates the anomaly detection time.

A comprehensive comparison of training and testing times highlights the efficiency of our model. In Table XV, existing GNN-based methods, such as E-GraphSAGE [7], GNN-based [15], and GraphDDoS [22], have training times of 3.1, 2.7, and 4.6 hours, respectively, while our GTAE-IDS model requires a slightly longer training time of 5.43 hours due to the inclusion of a transformer-based encoder. Although this increases the initial training time, it is a one-time investment that does not impact real-time performance. The additional complexity enhances feature extraction and detection robustness, justifying the extended training duration. However, as per MDT, which is crucial for real-time detection, GTAE-IDS demonstrates impressive performance. Our model achieves an MDT value of 1.28 μs, which is competitive with one and faster than other GNN-based methods, which have MDT values of 1.20 μs, 1.45 μs, and 2.12 μs, respectively. This shows that, despite the slightly higher training cost, our framework's real-time inference performance is comparable to or better than existing methods, confirming its suitability for near real-time anomaly detection.

In addition to conducting experiments on CIC-IDS2018 and ACI-IoT-2023 data sets, we have also deployed our framework in a real-world scenario to verify the time efficiency of GTAE-IDS. In this practical deployment, we use the same computing resources as in the testing phase (Intel Core i9-12950HX CPU and NVIDIA RTX A5500 GPU), representing a typical high-performance workstation in a cybersecurity environment. In this deployment scenario, we thoroughly assessed the latency, computational overhead, and efficiency of our GTAE-IDS framework. These factors are critical in evaluating whether our model meets the near real-time requirements for network anomaly detection. The scenario involved processing network traffic of 10,000 packets per second, simulating conditions typical of a medium-sized enterprise. Below is an expanded breakdown based on latency, computational overhead, and the practicalities of deployment.

Latency, defined as the time taken for the model to process each packet and produce an anomaly detection result, is a key factor in ensuring near real-time capability. In our testing, the MDT was measured at 1.32 μs per packet, demonstrating that GTAE-IDS introduces minimal latency. This low latency ensures timely anomaly detection without creating network bottlenecks, which is essential for maintaining smooth operations in security-sensitive applications. Specifically, the MDT value of 1.32 μs per packet (with 0.06 μs, 0.83 μs, and 0.43 μs allocated for GCT, GRLT, and ADT, respectively) enables GTAE-IDS to handle traffic rates of up to 10,000 packets per second without noticeable delay. This efficiency is attributed to sequential packet processing and early termination upon detecting anomalies. As a result, GTAE-IDS achieves near real-time detection with minimal latency and manageable computational overhead, making it well-suited for high-traffic environments, such as medium-sized enterprise networks.

In summary, although the transformer-based encoder requires an extended training phase, the deployed framework delivers robust and efficient real-time anomaly detection with minimal computational overhead. Achieving (mean) detection times of 1.28 μs per packet during testing and 1.32 μs per packet in deployment, along with early anomaly detection capabilities, GTAE-IDS offers a scalable and an efficient solution for near real-time applications in high-traffic network environments. These attributes underscore its effectiveness and reliability for timely intrusion detection in dynamic network conditions.

VI. Conclusion and Future Directions

In this paper, we proposed a graph transformer-based autoencoder framework for network intrusion detection. We demonstrated the effectiveness of GTAE-IDS in detecting anomalies in evolving network traffic, across various types of network attacks, and in different network environments, showcasing its potential for practical applications in network security. We evaluated our anomaly detection framework using the CIC-IDS2017 test graph sets and measured the performance based on TPR and FPR metrics. The results showed that as the dynamic graph representing network traffic evolved with more packets, the performance improved significantly. The best performance, with a TPR of 99.04%, was achieved by the ensemble anomaly detection model using a graph representation of first eight packets in a flow, indicating an early attack detection. Our research also assessed the performance of GTAE-IDS in detecting anomalies across various types of network attacks. Different graph sets were generated to represent specific attack types, and the results showed that GTAE-IDS consistently achieved high anomaly detection rates, exceeding 98.5%, across all attack types. The study further evaluated GTAE-IDS performance in detecting anomalies in a new environment using malicious traffic data from the CIC-IDS2018 data set. The results demonstrated that our framework maintained high anomaly detection rates (over 98%) even in a new network environment. The GRL paradigm integrated into our framework effectively captures both structural and contextual information within network traffic, resulting in enhanced accuracy for detecting malicious activities when compared to other state-of-the-art methods in the literature.

While the GTAE-IDS framework is designed to generalize to unseen data by modeling normal traffic patterns and identifying deviations, its performance is inherently tied to the stability of these patterns. Significant changes in traffic behavior over time, due to factors like temporal shifts or network variations, can impact anomaly detection accuracy. This limitation can be addressed through either periodically retraining the model with updated data to reflect evolving traffic patterns or implementing an adaptive mechanism that triggers retraining when key performance metrics indicate a decline. A promising direction for future research could also involve exploring hybrid approaches that combine periodic retraining with real-time adaptive mechanisms, allowing the model to dynamically adjust to evolving network conditions while minimizing computational overhead. Other promising future research directions include enhancing the scalability and adaptability of the GTAE-IDS framework for network intrusion detection by leveraging proprietary data sets. Additionally, integrating network vulnerability scanner data into the graph generation process presents an intriguing avenue for research, as it could enable even earlier detection of attacks by factoring in the exploitation probabilities of the machines involved in network communications.

Acknowledgment

The views and conclusions expressed in this article are those of the authors and do not reflect the official policy or position of U.S. Military Academy, U.S. Army, U.S. Department of Defense, or U.S. Government.

References

[1] C. Janiesch, P. Zschech, and K. Heinrich, "Machine learning and deep learning," *Electron. Mark.*, vol. 31, no. 3, pp. 685–695, 2021.

[2] Q. Xiao, J. Liu, Q. Wang, Z. Jiang, X. Wang, and Y. Yao, "Towards network anomaly detection using graph embedding," in *Proc. Int. Conf. Comput. Sci.*, Amsterdam, The Netherlands. Cham, Swizerland: Springer, Jun. 2020, pp. 156–169.

[3] J. Zhou, Z. Xu, A. M. Rush, and M. Yu, "Automating botnet detection with graph neural networks," 2020, *arXiv:2003.06344*.

[4] B. Zhang, J. Li, C. Chen, K. Lee, and I. Lee, "A practical botnet traffic detection system using GNN," in *Proc. Int. Symp. Cyberspace Saf. Secur.* Cham, Switzerland: Springer, 2021, pp. 66–78.

[5] W. W. Lo, G. Kulatilleke, M. Sarhan, S. Layeghy, and M. Portmann, "XG-BoT: An explainable deep graph neural network for botnet detection and forensics," *Internet Things*, vol. 22, Jul. 2023, Art. no. 100747.

[6] J. Zhao, X. Liu, Q. Yan, B. Li, M. Shao, and H. Peng, "Multi-attributed heterogeneous graph convolutional network for bot detection," *Inf. Sci.*, vol. 537, pp. 380–393, Oct. 2020.

[7] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp.*, Budapest, Hungary, Apr. 2022, pp. 1–9.

[8] L. Chang and P. Branco, "Graph-based solutions with residuals for intrusion detection: The modified E-GraphSAGE and E-ResGAT algorithms," 2021, *arXiv:2111.13597*.

[9] G. Hu, X. Xiao, M. Shen, B. Zhang, X. Yan, and Y. Liu, "TCGNN: Packet-grained network traffic classification via graph neural networks," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106531.

[10] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: Concepts, architectures, techniques, challenges, datasets, applications, and future directions," *J. Big Data*, vol. 11, no. 1, p. 18, Jan. 2024.

[11] L. Waikhom and R. Patgiri, "A survey of graph neural networks in various learning paradigms: Methods, applications, and challenges," *Artif. Intell. Rev.*, vol. 56, no. 7, pp. 6295–6364, Jul. 2023.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "A detailed analysis of the CICIDS2017 data set," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, Funchal-Madeira, Portugal. Cham, Switzerland: Springer, Jan. 2018, pp. 172–188.

[13] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, vol. 1, Jan. 2018, pp. 108–116.

[14] N. Bastian, D. Bierbrauer, M. McKenzie, and E. Nack, 2023, "ACI IoT network traffic dataset," IEEE Dataport, doi: 10.21227/qacj-3x32.

[15] D. Pujol-Perich, J. Suarez-Varela, A. Cabellos-Aparicio, and P. Barlet-Ros, "Unveiling the potential of graph neural networks for robust intrusion detection," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 49, no. 4, pp. 111–117, Jun. 2022.

[16] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," *ACM Trans. Privacy Secur.*, vol. 26, no. 3, pp. 1–36, 2023.

[17] Y. Cao, H. Jiang, Y. Deng, J. Wu, P. Zhou, and W. Luo, "Detecting and mitigating DDoS attacks in SDN using spatial–temporal graph convolutional network," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3855–3872, Nov. 2022.

[18] S. Govindaraju, W. V. R. Vinisha, F. H. Shajin, and D. A. Sivasakthi, "Intrusion detection framework using auto-metric graph neural network optimized with hybrid woodpecker mating and capuchin search optimization algorithm in IoT network," *Concurrency Comput., Pract. Exper.*, vol. 34, no. 24, p. 7197, Nov. 2022.

[19] A. Protogerou, S. Papadopoulos, A. Drosou, D. Tzovaras, and I. Refanidis, "A graph neural network method for distributed anomaly detection in IoT," *Evolving Syst.*, vol. 12, no. 1, pp. 19–36, Mar. 2021.

[20] J. Lan et al., "E-minBatch GraphSAGE: An industrial Internet attack detection model," *Secur. Commun. Netw.*, vol. 2022, pp. 1–12, Jul. 2022.

[21] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowl.-Based Syst.*, vol. 258, Dec. 2022, Art. no. 110030.

[22] Y. Li et al., "GraphDDoS: Effective DDoS attack detection using graph neural networks," in *Proc. IEEE 25th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2022, pp. 1275–1280.

[23] A. Premkumar, M. Schneider, C. Spivey, J. V. Pavlik, and N. D. Bastian, "Graph representation learning for context-aware network intrusion detection," *Proc. SPIE*, vol. 12538, pp. 82–92, Jun. 2023.

[24] L. F. Sikos, "Packet analysis for network forensics: A comprehensive survey," *Forensic Sci. Int., Digit. Invest.*, vol. 32, Mar. 2020, Art. no. 200892.

[25] *Libpcap*. Accessed: Oct. 1, 2024. [Online]. Available: https://www.tcpdump.org/

[26] B. Merino, *Instant Traffic Analysis With Tshark How-to*. Birmingham, U.K.: Packt Publishing, 2013.

[27] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.

[28] M. M. Alani, *Guide to OSI and TCP/IP Models*. Cham, Swizerland: Springer, 2014.

[29] Y. Yang, K. Zheng, B. Wu, Y. Yang, and X. Wang, "Network intrusion detection based on supervised adversarial variational auto-encoder with regularization," *IEEE Access*, vol. 8, pp. 42169–42184, 2020.

[30] Y. Dong et al., "HorusEye: A realtime IoT malicious traffic detection framework using programmable switches," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 571–588.

[31] G. Andresini, A. Appice, N. D. Mauro, C. Loglisci, and D. Malerba, "Multi-channel deep feature learning for intrusion detection," *IEEE Access*, vol. 8, pp. 53346–53359, 2020.

[32] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, Jul. 2006.

[33] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," 2020, *arXiv:2012.09699*.

[34] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 43, pp. 1–48, 2023.

[35] M. Goldstein and A. Dengel, "Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm," in *Proc. Poster Demo Track*, 2012, pp. 59–63.

[36] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, Dec. 2008, pp. 413–422.

[37] T. R. Bandaragoda, K. M. Ting, D. Albrecht, F. T. Liu, Y. Zhu, and J. R. Wells, "Isolation-based anomaly detection using nearest-neighbor ensembles," *Comput. Intell.*, vol. 34, no. 4, pp. 968–998, Nov. 2018.

[38] A. Bounsiar and M. G. Madden, "One-class support vector machines revisited," in *Proc. Int. Conf. Inf. Sci. Appl. (ICISA)*, May 2014, pp. 1–4.

[39] C. Canadian Institute for Cybersecurity. (2009).*NSL KDD Dataset*. Accessed: Aug. 19, 2023. [Online]. Available: https://www.unb.ca/cic/datasets/nsl.html

[40] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.

[41] H. Hindy et al., "A taxonomy of network threats and the effect of current datasets on intrusion detection systems," *IEEE Access*, vol. 8, pp. 104650–104675, 2020.

[42] S. Hore, J. Ghadermazi, D. Paudel, A. Shah, T. K. Das, and N. D. Bastian, "Deep PackGen: A deep reinforcement learning framework for adversarial network packet generation," 2023, *arXiv:2305.11039*.

[43] J. Ghadermazi, A. Shah, and N. D. Bastian, "Towards real-time network intrusion detection with image-based sequential packets representation," *IEEE Trans. Big Data*, vol. 11, no. 1, pp. 157–173, Feb. 2025.

[44] C. University of New Brunswick. (2023).*Canadian Institute for Cybersecurity*. [Online]. Available: https://www.unb.ca/cic/

[45] I. of Things Res. Lab (IoTRL). (2023).*Army Cyber Institute (aci)*. [Online]. Available: https://cyber.army.mil/Research/Research-Labs/Datasets/\#dataset1

[46] B. A. Pratomo, P. Burnap, and G. Theodorakopoulos, "Unsupervised approach for detecting low rate attacks on network traffic with autoencoder," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Services (Cyber Security)*, Jun. 2018, pp. 1–8.

[47] W. Wang et al., "HAST-IDS: Learning hierarchical spatial–temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.

[48] H. Liu, B. Lang, M. Liu, and H. Yan, "CNN and RNN based payload classification methods for attack detection," *Knowl.-Based Syst.*, vol. 163, pp. 332–341, Jan. 2019.

[49] E. L. Goodman, C. Zimmerman, and C. Hudson, "Packet2 Vec: Utilizing word2 Vec for feature extraction in packet data," 2020, *arXiv:2004.14477*.

[50] M. Hassan, M. E. Haque, M. E. Tozal, V. Raghavan, and R. Agrawal, "Intrusion detection using payload embeddings," *IEEE Access*, vol. 10, pp. 4015–4030, 2022.

[51] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.



Jalal Ghadermazi is currently pursuing the Ph.D. degree with the Industrial and Management Systems Engineering Department, University of South Florida, specializing in the field of cybersecurity. His research interests span the areas of robustness, real-time analytics, and adversarial machine learning in cybersecurity applications. He integrates principles from computer science, operations research, data science, and information technology to develop innovative solutions for enhancing cybersecurity.



Soumyadeep Hore (Graduate Student Member, IEEE) received the B.Tech. degree in mechanical engineering from West Bengal University of Technology, Salt Lake, Kolkata, India, in 2015, and the M.Tech. degree in industrial engineering from IIT (ISM) Dhanbad, India, in 2019. He is currently pursuing the Ph.D. degree with the Department of Industrial and Management Systems Engineering, University of South Florida, Tampa, FL, USA. His research interests include cybersecurity, predictive and prescriptive analytics, decision making under uncertainty, machine learning, multi-objective optimization, and deep reinforcement learning.



Ankit Shah (Senior Member, IEEE) is currently an Assistant Professor in operations and decision technologies with the Kelley School of Business, Indiana University, and the Deputy Director of Cybersecurity with the Kelley's Data Science and Artificial Intelligence Laboratory (DSAIL). His research interests are at the intersection of computer science, operations research, data science, and information technology, with a strong focus on artificial intelligence (AI) for cybersecurity and defense applications and security of AI systems. His current research work is in the area of deep reinforcement learning and machine learning for secure and efficient systems.



Nathaniel D. Bastian (Senior Member, IEEE) is an Assistant Professor with the Department of Mathematical Sciences, United States Military Academy, also serving as a Chief Scientist and the Director, Office of Science and Engineering at the Army Cyber Institute, West Point. His research interests lie at the intersection of artificial intelligence (AI), operations research, data science, and systems engineering with a strong focus on AI security, robustness and resiliency, intelligent battlefield command and control, and AI for cyberspace operations.